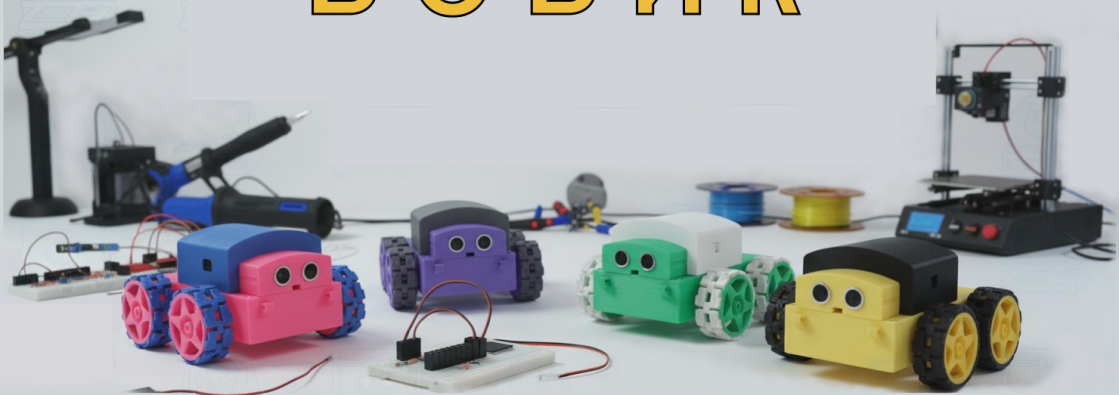


Инструкция по сборке робота

БОБИК



АЭРОЛОГОС

Инструкция по сборке робота Бобик

aerologos.by

31 марта 2026 г.

Оглавление

1	Введение	3
2	Техника безопасности	4
3	Комплектующие	6
4	Требуемые инструменты	8
5	Первичная сборка робота	9
5.1	Пайка двигателей	9
5.2	Пайка гребенки к драйверу	10
5.3	Пайка разъемов	11
5.4	Сборка шасси	13
5.5	Установка контроллера	22
5.6	Установка датчика	24
6	Программирование робота	27
6.1	Загрузка исходного кода	28
6.2	Загрузка среды разработки	29
6.3	Программирование робота	30
6.4	Компиляция, загрузка и тестирование	32
7	Финальная сборка робота	36
7.1	Сборка фургона	36

7.2	Нарезка стержней	36
7.3	Робот на колесной базе	36
7.4	Робот на гусеничной базе	38
7.5	Установка аккумулятора	39
8	Продвинутые алгоритмы движения	40
8.1	Движение с поворотами	40
8.2	Движение с поиском пути	40
9	Погружение в детали	42
9.1	Шасси	43
9.2	Измерение расстояния	44
10	Отладка	46
11	Рекомендации	49
12	Заключительное слово	50

Глава 1

Введение

Перед вами руководство по сборке, настройке и программированию робота Бобик на компонентной базе Arduino - Вашего будущего цифрового домашнего питомца, друга и платформы для испытаний.

Наш робот это детально проработанная и доведенная до ума идея. Мы прошли опыт сборки стандартных конструкторов и выявили обширный список недостатков, которые удалось устранить, всерьез поработав над этим проектом.

Мы упростили задачу по сборке и обслуживанию робота. Придали конструктору форму и идентичность, что делает его не только инструментом развлечения но и приличной декорацией интерьера. Не забыли и про масштабирование данного решения, поэтому робот обладает крепежом и отверстием для подключения дополнительного оборудования.

Он не боится переноски в рюкзаке или сумке, перевозки в машине, заполненной вещами. Он может послужить отличным подарком на день рождения ребенку или коллеге по работе.

Наверное достаточно слов. Давайте начнем...

Глава 2

Техника безопасности

ВНИМАНИЕ

Меры предосторожности при пайке! Конструктор предполагает пайку внутренних коммуникаций. Поэтому Вам следует работать в хорошо вентилируемом помещении, надевать защитные очки, перчатки и спецодежду. Рабочее место должно быть чистым и хорошо освещенным, а электроинструменты - в исправном состоянии. Также нельзя работать рядом с легковоспламеняющимися материалами и курить. Настоятельно рекомендуется паять при помощи штатива "3-я рука" так как он значительно сокращает риск получения ожогов и существенно упрощает процесс пайки. Также вам не помешает коробка с металлической стружкой для чистки жала паяльника.

СОВЕТ

Эксплуатация никелевых аккумуляторов! В комплекте прилагается никелевый аккумулятор. Стоит помнить о том, что такой тип батарей имеет свойство "запоминать" стартовый уровень тока, при котором батарея была поставлена на зарядку. Это приводит к тому, что заряженная батарея может отдавать ток только до уровня, с которого была начата зарядка, и затем будет вести себя, как разряженная. Такие батареи предпочтительно разряжать, как можно сильнее, перед зарядкой.

Глава 3

Комплектующие

Перед началом работы разложите все детали на столе. Убедитесь, что у вас есть:

- Вкладыш для обратной связи с нами
- Шасси
- Крышка шасси
- Голова
- Фургон
- Дверь фургона
- Крышка аккумуляторного отсека
- 4 колеса
- 2 вала
- 40 единиц протектора
- 80 сантиметров пластиковой проволоки

- 6 коротких болтиков
- 2 средних болтика
- 2 длинных болтика
- 10 гаек
- Выключатель
- 2 разъема под выключатель
- Никелевый аккумулятор "Крона"
- Разъем под аккумулятор
- Драйвер двигателей L298n mini
- 2 двигателя с редуктором
- 2 конденсатора 104
- Гребенка штыревая с 10 пинами
- Контроллер форм-фактора Arduino UNO
- Синий кабель для прошивки контроллера
- Ультразвуковой дальномер HC-SR04
- 10 проводов Dupont папа-мама
- 2 провода Dupont мама-мама
- 1 провод Dupont папа-папа

Глава 4

Требуемые инструменты

Вам потребуется следующий набор инструментов:

- Компьютер
- Паяльник
- Штатив "3-я рука"
- Медная губка для чистки жала (желательно)
- Флюс
- Припой
- Крестообразная отвертка
- Бокорезы или ножницы
- Плоскогубцы (желательно небольшие)
- Клей-пистолет или супер клей

Глава 5

Первичная сборка робота

5.1 Пайка двигателей

В первую очередь потребуется разобраться с проводами и убедиться, что везде все припаяно, как следует.

Достаньте из набора 2 двигателя, 2 конденсатора 104, 2 провода мама-мама разного цвета. Разрежьте оба провода ровно посередине, оголите полученные концы каждого провода и наматывайте их поочередно на ножки конденсатора. На каждом конденсаторе должно быть по 2 провода разного цвета. См. Рис. 5.1.

Перед пайкой необходимо все залудить. Лужение — это нанесение тонкого слоя расплавленного припоя на металлическую поверхность с целью упростить спаивание ее с другой поверхностью.

Нагрейте паяльник, затем нанесите флюс на обмотанные ножки конденсатора и контакты электродвигателей, чтобы пайка прошла без сложностей.

Поочередно расплавьте паяльником достаточное количество припоя и произведите лужение каждого контакта двигателей, затем ножек конденсаторов до того, как начнете спаивать все

вместе.

У Вас должен получиться примерно следующий результат:

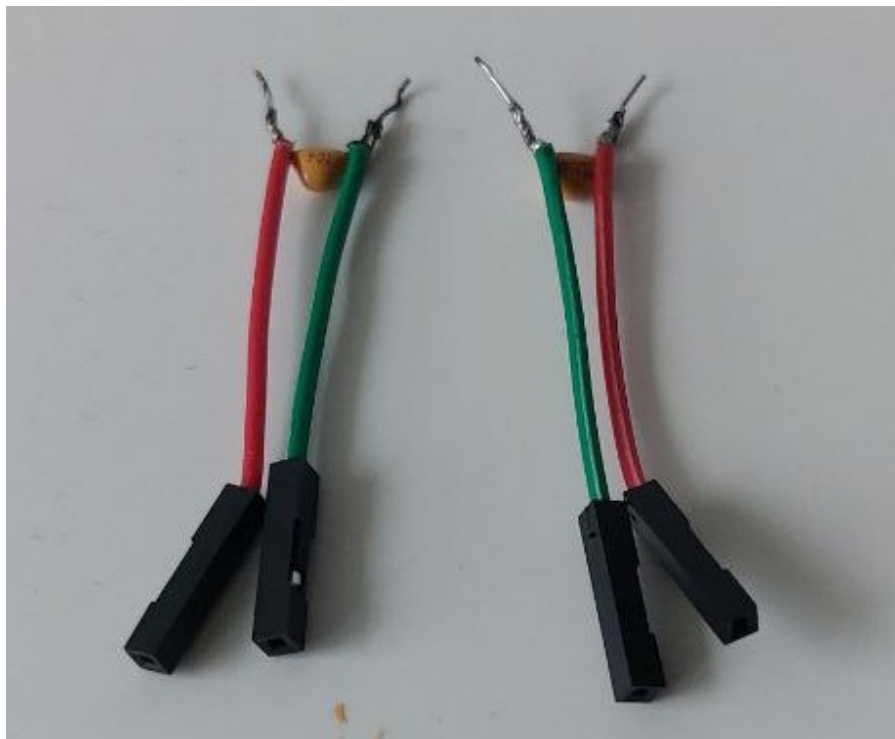


Рис. 5.1: Провода с конденсаторами.

Теперь необходимо припаять полученный результат к двигателям. Постарайтесь чтобы пайка была зеркальной. Например, если вы выбрали 1 провод красного и один зеленого, то, при размещении моторов проводами вверх, провода одного цвета должны быть оба внутри. Провода другого - снаружи. Смотрите пример на Рис. 5.2:

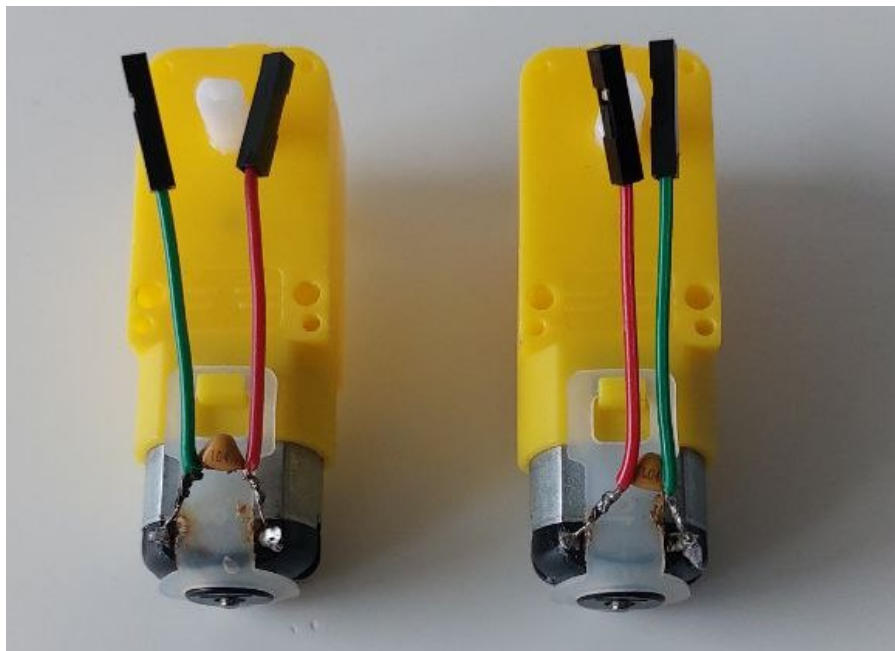


Рис. 5.2: Двигатели с проводами.

5.2 Пайка гребенки к драйверу

Возьмите со стола гребенку и разделите ее на 5 частей по 2 пина в каждой.

Вставьте первую пару в отверстия $+$ - на драйвере и закрепите в перевернутом состоянии в штативе, таким образом,

чтобы пальцы штатива держали вместе и гребенку и драйвер, позволяя легко сочленить их припоем.

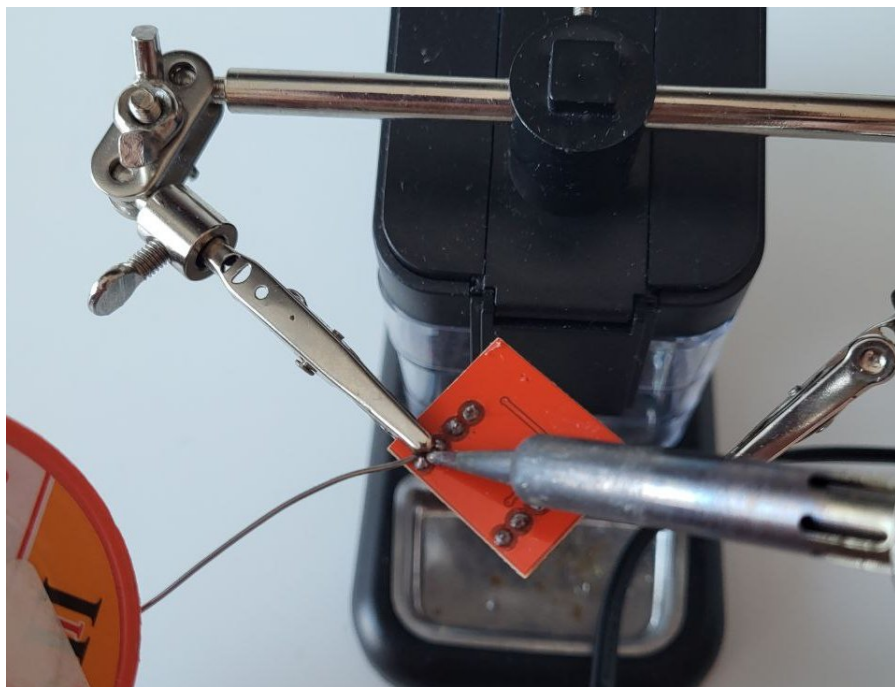


Рис. 5.3: Пайка гребенки к драйверу.

Нанесите флюс в местах соединения и произведите пайку, одновременно поднося проволоку с припоем и жало паяльника к мастам пайки. Повторите операцию для остальных 4-х пар гребенки, припаяв их к соответствующим отверстиям драйвера.

На выходе должно получиться достаточно неплохое для домашних условий качество пайки. Если же Вы предпочитаете все доводить до совершенства, то воспользуйтесь феном для пайки.

5.3 Пайка разъемов

Теперь необходимо припаять все имеющиеся разъемы к проводам, которые будут отвечать за подачу питания на плату управления роботом.

Возьмите первый разъем из набора и припаяйте его к красному проводу разъема для "кроны". Затем обожмите изолированную часть провода плоскогубцами, придавливая специализированные выступы. См. Рис. 5.4.

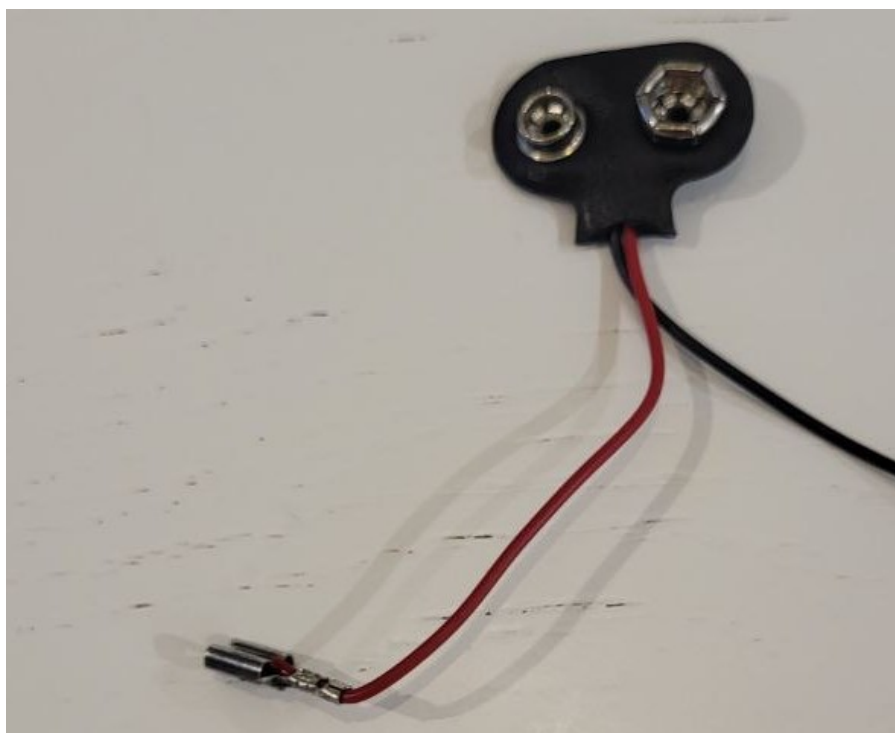


Рис. 5.4: Разъем на проводе для кроны.

Второй такой разъем припаяйте к одному из концов провода типа папа-папа. Другой конец этого провода можно сразу

согнуть на 110-120 градусов, чтобы потом его было легко подключить к плате Arduino.

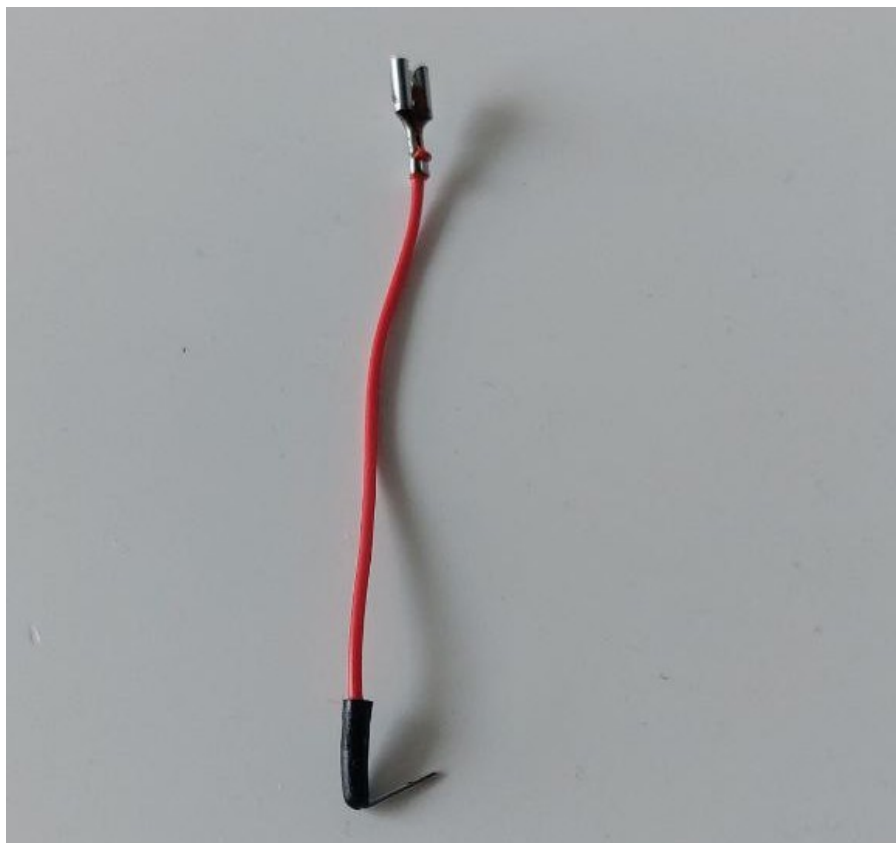


Рис. 5.5: Разъем на проводе папа-папа.

Последним местом пайки будет соединение черного провода от разъема для кроны с проводом, который будет обеспечивать соединение "земли" между Arduino и драйвером двигателей. Оголите черный провод. Возьмите провод типа папа-мама из набора и обмотайте выступ "папа" оголенной частью черного провода разъема под аккумулятор. См. Рис. 5.6

Затем нанесите флюс на полученное соединение и спаяйте его. Для большей надежности оголенные части проводов земли можно обмотать изолентой или наложить термоусадку, чтобы повысить надежность соединения от разрыва:

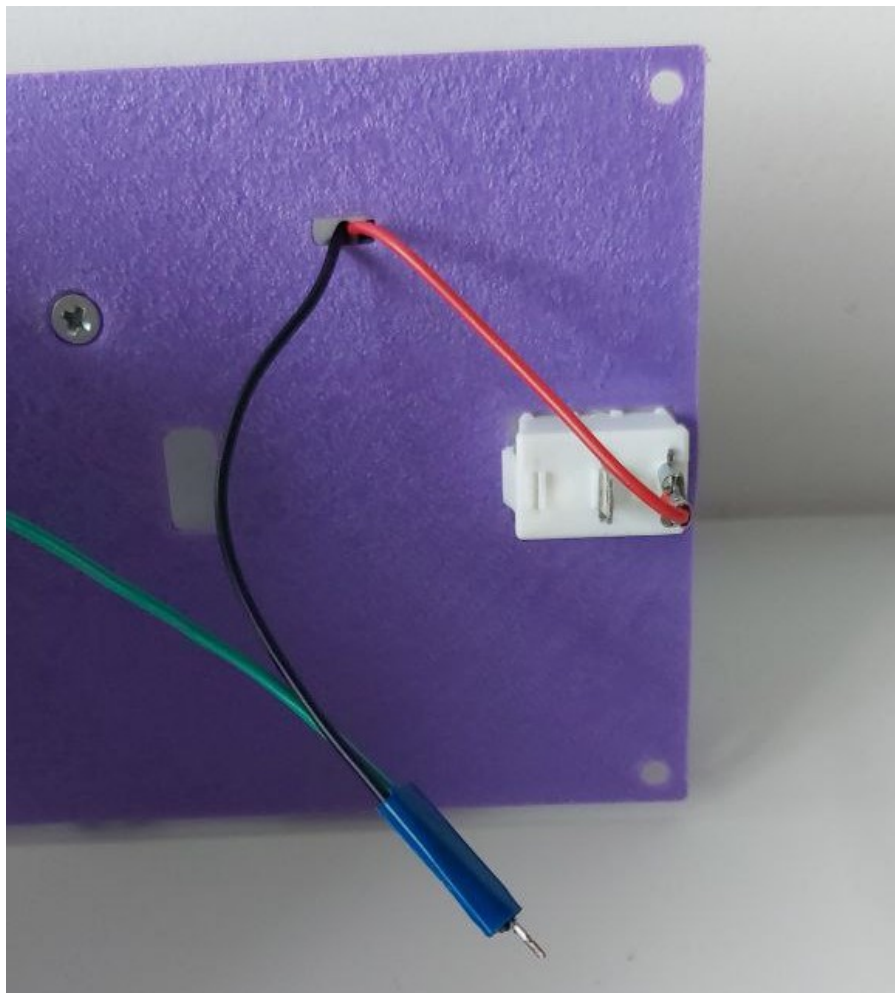


Рис. 5.6: Земля для Arduino и драйвера.

На этом все операции, связанные с пайкой, закончены. Не забудьте выключить паяльник!

5.4 Сборка шасси

Далее Вам потребуется установить драйвер двигателей в шасси робота. Для этого внутри имеется специализированная выемка. Рекомендуется закрепить драйвер в ней для эстетичности. Используйте для этого термоклей (клей-пистолет) или супер клей. Нанесите клей по контурам выемки в корпусе и вставьте чип. Должно получиться примерно следующее:



Рис. 5.7: Установка драйвера двигателей.

ЗАМЕТКА

На драйвере имеется 10 контактных точек.

- **2 (+ и -)** - для подачи напряжения
- **4 (INT 1 - 4)** - для подачи сигналов с контроллера на двигатели, чтобы можно было менять направление вращения двигателей (вперед или назад)
- **4 (MOTOR A - B)** для подачи напряжения с драйвера на двигатели

Давайте для начала соединим двигатели с драйвером. Установите двигатели в предназначенные для них позиции внутри корпуса для шасси и закрепите длинными болтами из набора.

Подключите провода в соответствии с данными из таблицы или руководствуясь изображением на Рис. 5.8. Провода двигателей подключаются к точкам с надписями MOTOR-B, MOTOR-A. Отсчет ведется слева-направо. Не перепутайте с входами INT1 - 4 так как они необходимы для приема сигналов с контроллера.

	левый	правый
верхний	1	3
нижний	2	4

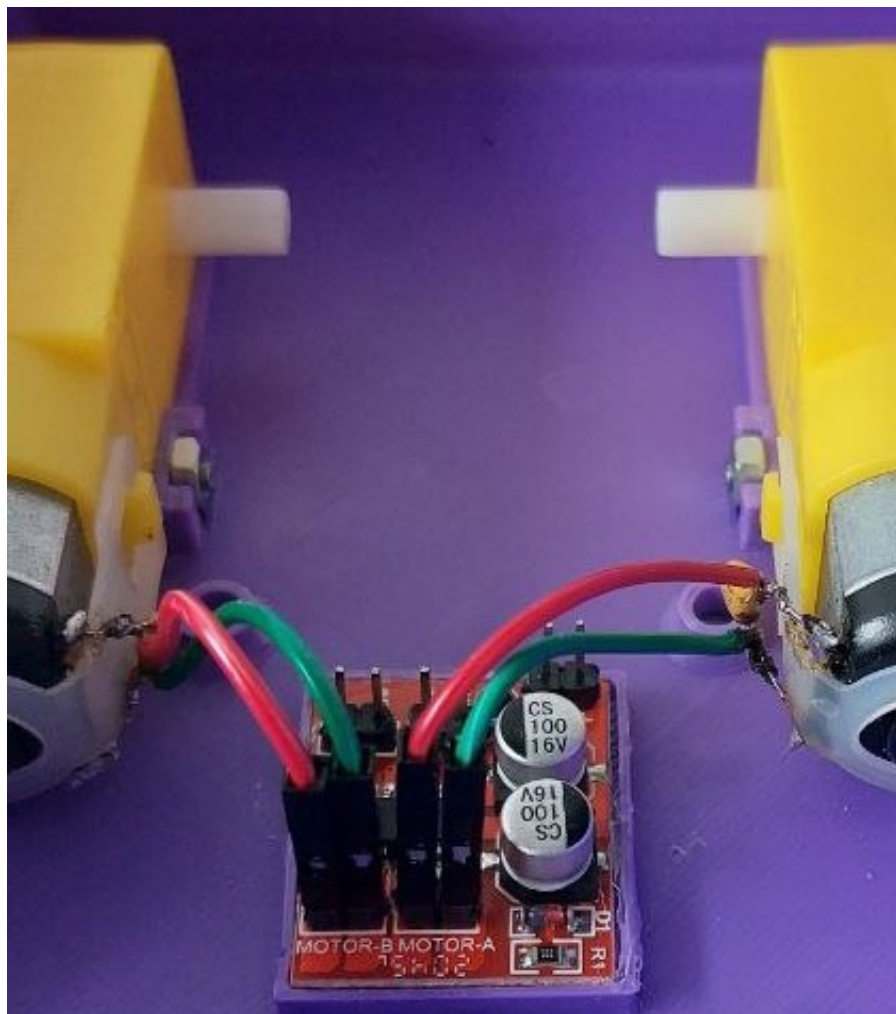


Рис. 5.8: Подключение двигателей к драйверу.

Возьмите крышку для шасси и установите в нее переключатель в специализированное отверстие сзади, как показано на Рис. 5.9:

- 1 - вперед
- 0 - назад



Рис. 5.9: Переключатель в корпусе.

Протяните провода разъема для кроны через отверстие в коробке для батареи. Рис. 5.10.

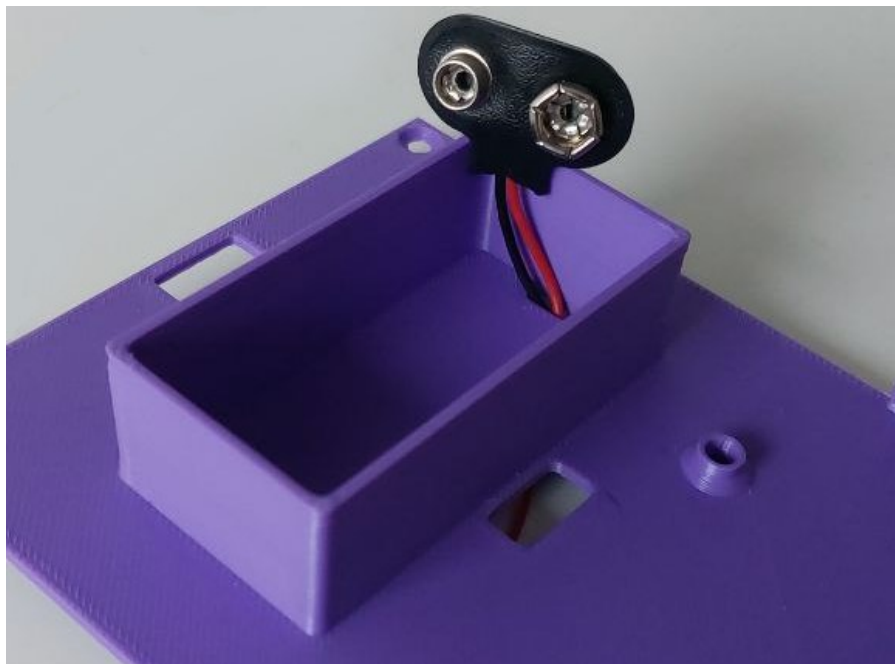


Рис. 5.10: Разъем для кроны в корпусе.

Подключите красный провод к одному из посадочных мест переключателя. К другому подключите красный провод папа-папа с припаянным разъемом, который мы подготовили ранее. Рис. 5.11.

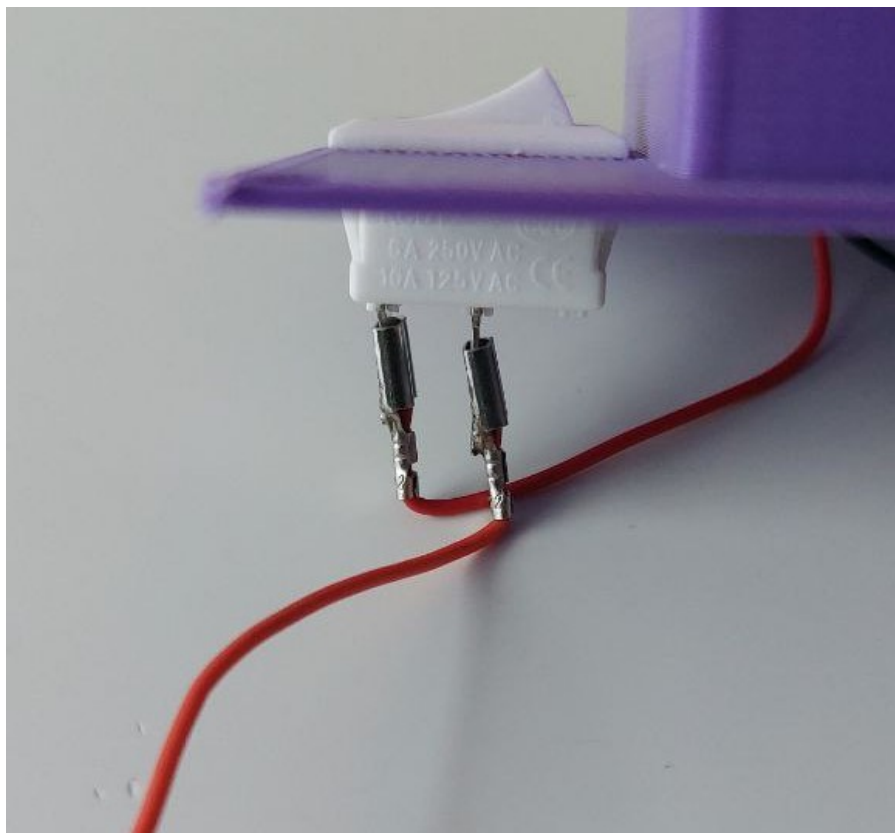


Рис. 5.11: Переключатель с проводами.

Установите валы из набора в корпусе шасси в позиции, расположенные сзади. Возьмите 1 провод папа-мама и подключите его к разъему со знаком плюс (+) на драйвере двигателей. Затем подключите провод для общей земли, который мы соединили с землей аккумулятора, и подключите его там же к разъему со знаком минус (-).

Далее возьмите еще 4 провода папа-мама и подключите их к 4-м входам драйвера (INT1 - 4). Смотрите Рис. 5.12.

И не забудьте вставить 4 гайки из набора в соответствующие отверстия сверху шасси. К ним мы впоследствии прикрутим крышку. Если при наклоне шасси гайки выпадают, закрепите их термоклеем или густой смазкой типа "солидол".

На этом сборку шасси робота можно считать законченной.

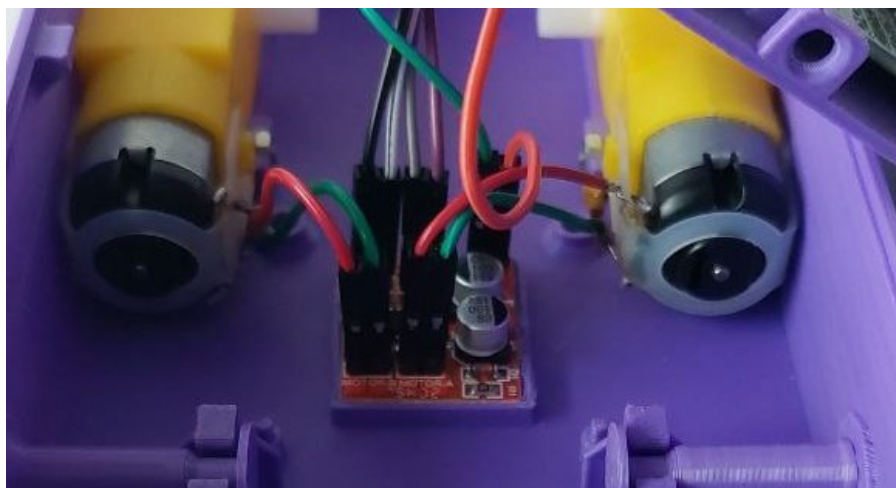


Рис. 5.12: Шасси в сборе.

5.5 Установка контроллера

Перейдем к установке контроллера. Расположите плату UNO на крышке корпуса, установив ее в предназначенные для этого выступы, расположенные по диагонали, и закрепите 2 болтами средней длины из набора, просунув их через 2 отверстия в крышке. Смотрите Рис. 5.13.



Рис. 5.13: UNO в корпусе.

Протяните один из проводов переключателя и провода, выходящие из драйвера двигателей, через заднее отверстие в крышке для шасси и подключите их к плате UNO, пользуясь данными из следующей таблицы:

Провод	Разъем UNO
плюс от переключателя	VIN
плюс для драйвера	5V
минус разъема аккумулятора и драйвера	GND
INT1	D2
INT2	D4
INT3	D7
INT4	D8

Чтобы избежать отрыва проводов от драйвера при соединении их с платой Arduino рекомендуется согнуть концы "папа" на необходимый угол и только после этого вставлять их в соответствующие гнезда.

После подключения всех проводов накиньте крышку на корпус шасси и затяните ее 4 короткими болтами из набора, вкручивая их в гайки, заложенные в корпус ранее.

5.6 Установка датчика

Возьмите "голову" робота и вставьте в соответствующее положение ультразвуковой датчик HC-SR04. Закрепите датчик термо- или супер клеем. Также убедитесь, что гребенка контактов смотрит от датчика, а не вниз. Если это не так, воспользуйтесь плоскогубцами и выровняйте контакты по одному, стараясь не сломать их.



Рис. 5.14: Ультразвуковой датчик в корпусе.

Возьмите 4 провода папа-мама из набора и подключите их к датчику. А затем соедините с платой UNO, используя данные из следующей таблицы:

Разъем датчика	Разъем UNO
VCC	3.3V
GND	GND
TRIG	13
ECHO	12

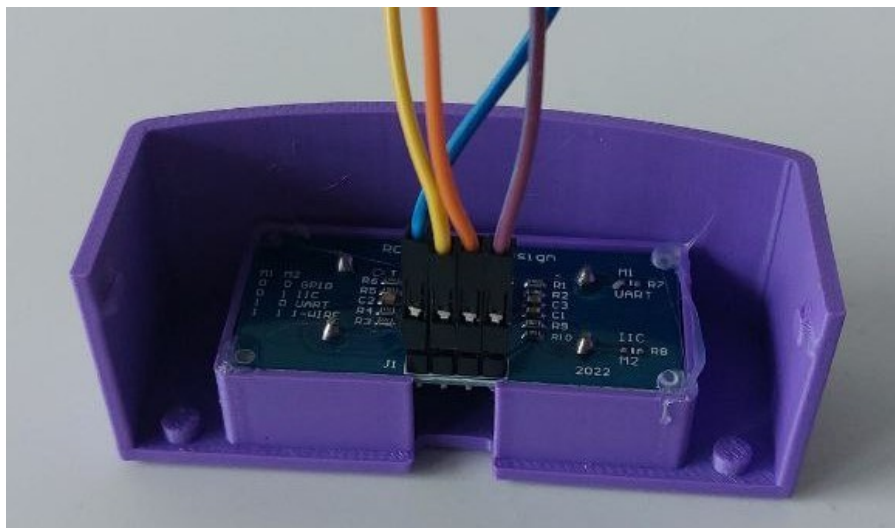


Рис. 5.15: Ультразвуковой дальномер с проводами.

Закрепите голову на корпусе, сначала вставив передние выступы внутри головы в корпус, а затем защелкните задние.

Еще не все детали соединены друг с другом. Однако не стоит торопиться, так как в первую очередь необходимо убедиться, что все собрано правильно и управляющая программа соответствует сборке. На данном этапе ваш робот должен выглядеть вот так:

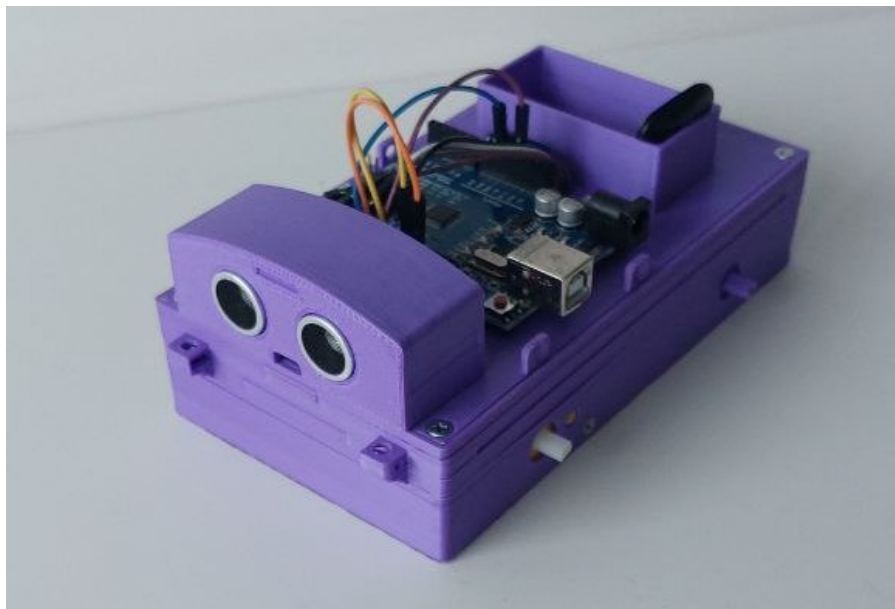


Рис. 5.16: Робот перед программированием.

Глава 6

Программирование робота

Для успешной работы робота Вам потребуется исполняющая программа. Если вы имеете опыт программирования UNO или привыкли делать все сами, то можете попробовать написать всю логику движения робота самостоятельно.

Для менее же опытных пользователей мы предлагаем воспользоваться нашими наработками и скачать готовую программу. Код этой программы испытан на практике и хорошо зарекомендовал себя.

Также мы не хотим заставлять Вас копировать код вручную, так как ручной набор занимает много времени и такая практика сопряжена с ошибками.

Давайте пока просто скачаем его и поработаем с готовыми сценариями. Позже, разобравшись в логике, Вы сможете усовершенствовать их на свое усмотрение.

6.1 Загрузка исходного кода

Наберите в браузере следующий адрес для скачивания программы: `https://github.com/aerologos/ardu_bot`

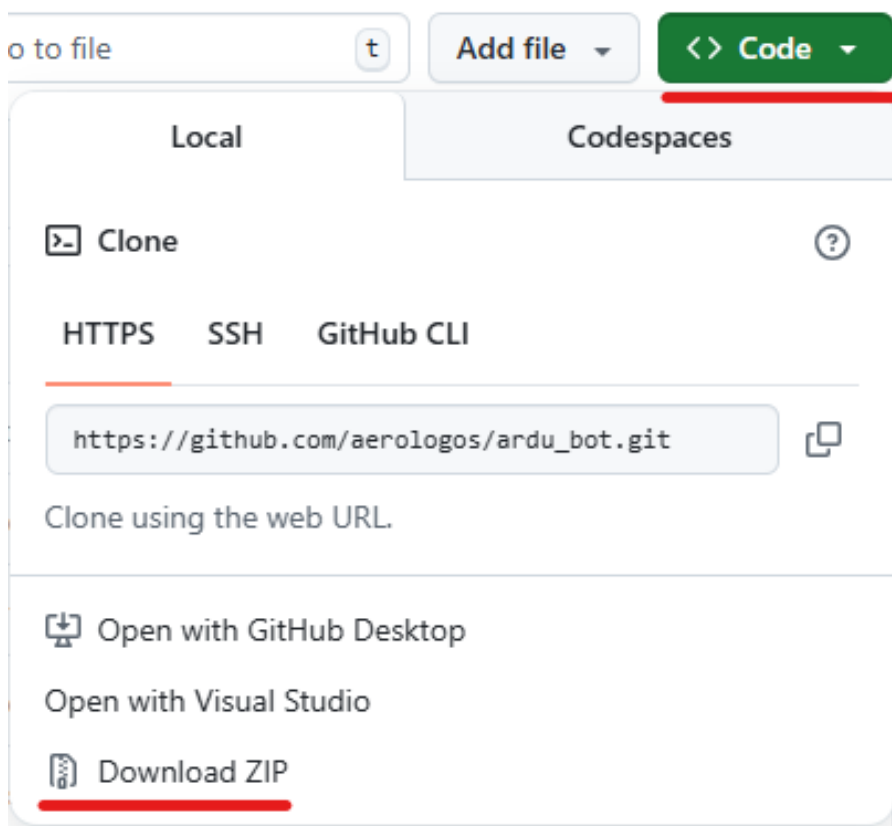


Рис. 6.1: Загрузка исходного кода с github.

Если вы опытный пользователь git, то загрузите себе исходный код на компьютер удобной программой для работы с этой системой контроля версий.

Если же вы в первый раз посещаете сайт github, то пока отложите изучение git на потом, а сейчас мы предлагаем просто

загрузить архив исходного кода в файловую систему Вашего компьютера.

Затем нажмите последовательно на **Code->Download ZIP**, как показано Рис. 6.1

После скачивания распакуйте архив и переместите полученную папку с файлами в удобное место хранения ваших файлов. Например в директорию **Projects** на жестком диске.

СОВЕТ

Если на компьютере не установлен архиватор, настоятельно рекомендуем скачать и установить такой. Например **7Zip**.

6.2 Загрузка среды разработки

Для того, чтобы загрузить полученный исходный код в 'мозги' Вашего робота потребуется среда разработки и загрузки **Arduino IDE**.

Наберите в поисковике в браузере **Скачать Arduino IDE** и загрузите себе установочный файл данной программы.

После загрузки произведите установку среды на ваш компьютер.

Если вы пользователь Linux или MacOS просто следуйте инструкциям по установке среды на сайте разработчика.

6.3 Программирование робота

Запустите установленную **Arduino IDE**. Последовательно нажмите **File -> Open** и в открывшемся окне укажите путь к файлу `ardu_bot.ino` из директории с загруженным исходным кодом:

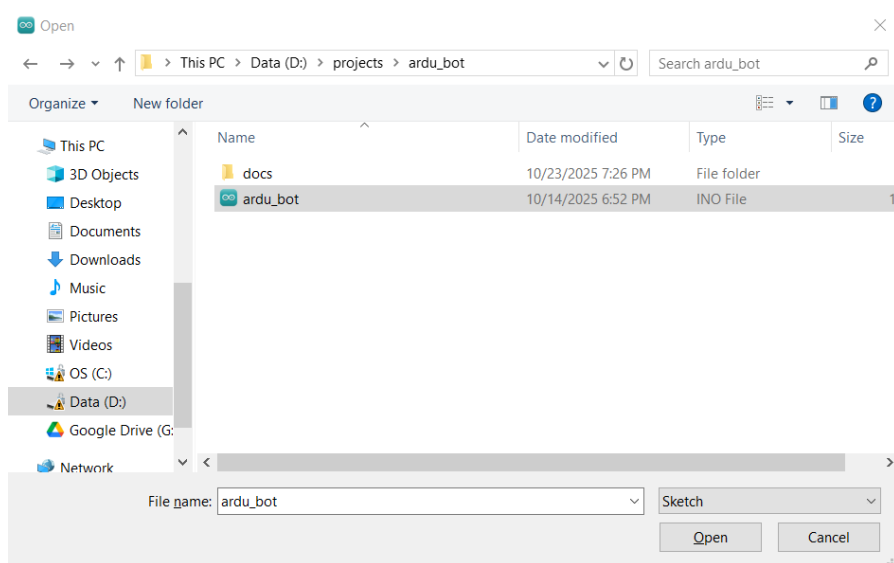


Рис. 6.2: Открытие прошивки в Arduino IDE.

Выполнив все предыдущие действия, перед Вашими глазами должен открыться листинг головного файла программы для программирования Arduino чипов, представленный на Рис. 6.3.

Мы подготовили несколько алгоритмов управления роботом, переключаясь между которыми Вы можете убедиться в работоспособности вашей сборки. Если робот не движется или движется не в соответствии алгоритму, значит при сборке были допущены ошибки и Вам необходимо будет исправить их.

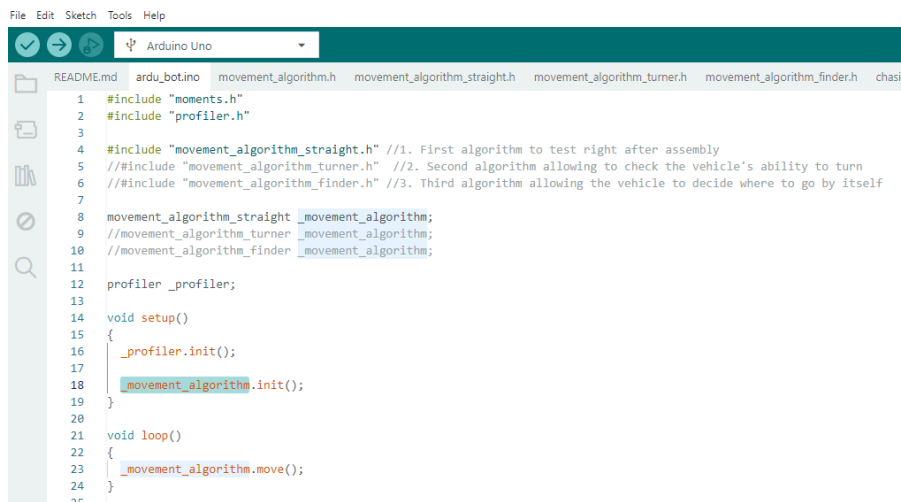


Рис. 6.3: Содержание файла ardu_bot.ino

Каждый алгоритм представлен группой из 2 строк в файле *ardu_bot.ino*. Пример:

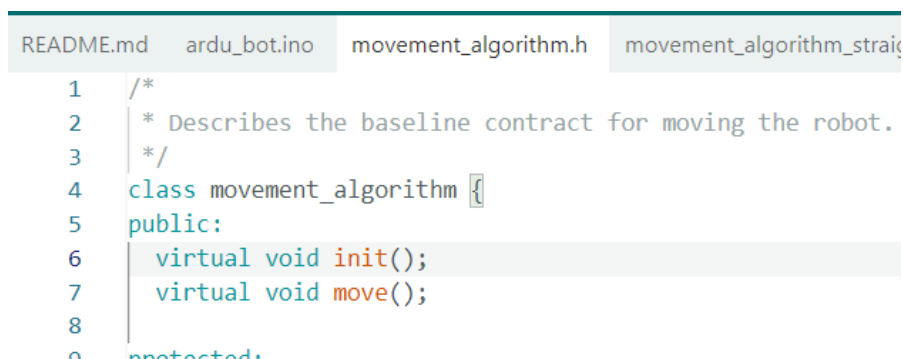
- `#include "movement_algorithm_straight.h"`
- `movement_algorithm_straight _movement_algorithm;`

Первая строка отвечает за объявление библиотеки, в которой находится алгоритм, вторая - за создание объекта исполняемого алгоритма. Объект - это сущность, которую пока для простоты можно представить, как действующего персонажа, у которого есть набор команд, вызывая которые мы можем требовать от него их выполнить.

Другие алгоритмы по умолчанию выключены. Для этого используется стандартный для программирования набор символов - двойной слэш `//`. Это необходимо по той причине, что во первых нельзя одновременно активировать более 1 библиотеки, имеющей одного предка, во вторых имя объекта для всех

алгоритмов указано одинаковым, чтобы уменьшить количество действий при переключении между ними.

Алгоритмы имеют общего предка `movement_algorithm`, который определяет контракт для всех наследников. Наследники, реализующие данный контракт, могут быть использованы одинаково, что и продемонстрировано в файле `ardu_bot.ino`. В то же время каждый из наследников предоставляет свой уникальный вариант исполнения алгоритма. И таких вариантов можно создавать бесконечное множество, не изменяя ничего в уже существующих, а просто добавляя и переключаясь между ними. Такая возможность в программировании называется масштабированием.



```
1  /*
2  * Describes the baseline contract for moving the robot.
3  */
4  class movement_algorithm {
5  public:
6      virtual void init();
7      virtual void move();
8
9  protected:
```

Рис. 6.4: Базовый класс для алгоритмов движения.

Еще один немаловажный объект в нашем головном файле, это `_profiler`. По коду других файлов разбросано его использование, а вызов метода `init` в головном файле активирует его. Он необходим для отладки, когда робот ведет себя не так, как запланировано и нужно выяснить причину. Если в его использовании нет необходимости, вызов `init` можно закомментировать через упомянутый ранее двойной слэш. Но об этом чуть позже.

6.4 Компиляция, загрузка и тестирование

Проверим работоспособность нашего робота. По умолчанию активирован алгоритм, в котором робот просто движется вперед до ближайшего препятствия, после чего отъезжает назад и снова едет вперед, пока не упрется в стену. Он должен это повторять пока его не выключат.

Этот алгоритм необходим для того чтобы убедиться, что робот собран без ошибок, двигатели крутятся в правильном направлении, а дальномер видит препятствия.

Ваш робот пока без колес и это нормально. Не торопитесь их надевать.

Откройте Arduino IDE и нажмите кнопку компиляции, чтобы убедиться, что код правильный и не содержит ошибок. Рекомендуется это делать каждый раз, когда вы вносите изменения, до попытки прошить новым кодом контроллер.

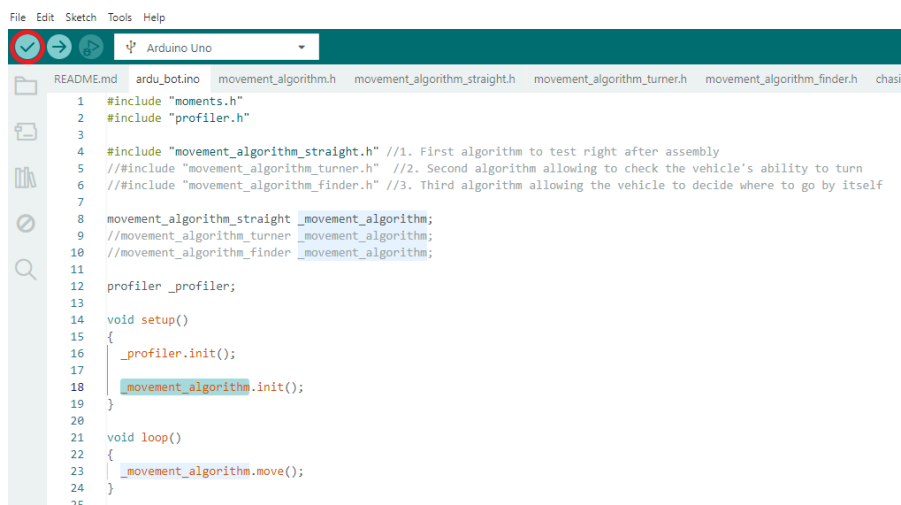


Рис. 6.5: Компиляция исходного кода.

Если все в порядке, то среда выдаст сообщение *Done compiling!*, означающее что в коде нет ошибок.

Возьмите толстый синий провод из набора, подключите один из его концов к компьютеру, другой - к контроллеру.

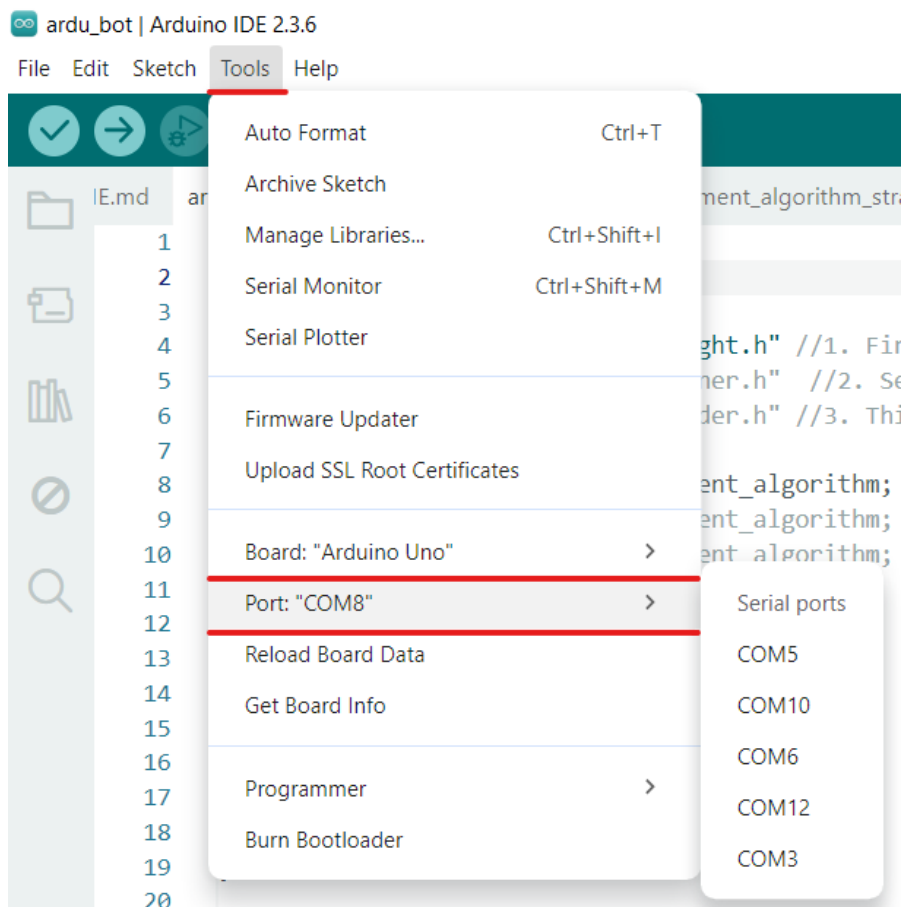


Рис. 6.6: Настройка Arduino IDE для загрузки кода.

В верхнем меню Arduino IDE нажмите *Tools -> Board -> Arduino AVR Boards -> Arduino UNO*, чтобы выбрать тип

контроллера, соответствующий установленному в работе.

Затем *Tools* -> *Port* -> *COM N*, где N - номер порта в Вашем ПК, в который вставлен синий провод. Если непонятно, какой это номер, просто попробуйте увидеть разницу в списке отображающихся в Arduino IDE портов, отсоединяя и присоединяя синий провод к компьютеру. Тот, что появляется и исчезает и есть нужный вам порт.

Нажмите на кнопку загрузки кода на контроллер:

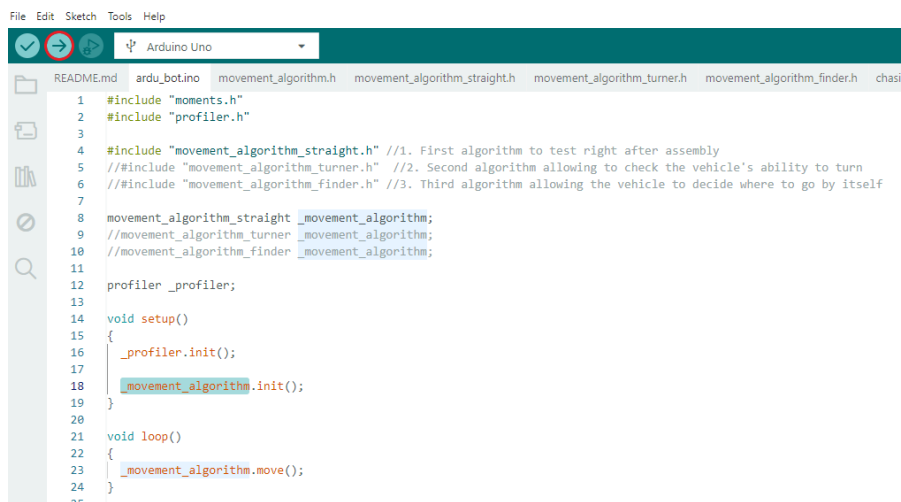


Рис. 6.7: Загрузка кода на контроллер.

Дождитесь последовательно появления 2 сообщений:

- *Done compiling!*
- *Done uploading!*

Вытащите синий провод из контроллера, поставьте робот на стол и переведите выключатель в активное положение - оба двигателя должны начать вращаться вперед.

Поставьте руку прямо перед глазами робота - оба двигателя должны начать вращаться назад.

Если это не так, вернитесь в начало данной инструкции и последовательно проверьте правильность соединений каждого узла.

Если все так, как написано, значит Вы все сделали правильно и можно выключить перекключатель и перейти к финальной сборке робота.

Глава 7

Финальная сборка робота

7.1 Сборка фургона

Возьмите фургон робота, вставьте 2 гайки в специальные емок-сти, куда будет прикручиваться задняя стенка. Накиньте фургон на шасси с одной стороны, вставив выступы фургона в пазы в шасси. Затем защелкните крышу с другой стороны.

Вставьте стенку фургона сзади и закрутите 2-мя мелкими болтами из набора.

7.2 Нарезка стержней

Теперь необходимо нарезать пластмассовую проволоку длиной 80 см на короткие стержни длиной 2 см каждый. Старайтесь соблюдать эту длину, чтобы Вам хватило ровно на 40 стержней и не пришлось потом искать альтернативу.

Затем нужно решить, какая платформа вам интересна: колесная либо гусеничная.

7.3 Робот на колесной базе

Разделите звенья и стержни на 4 части - по 10 звеньев и 10 стержней для каждого колеса.

Соедините каждую группу звеньев стержнями таким образом, чтобы осталось соединить только образовавшиеся 2 конца цепи. Оберните полученной цепью колесо и замкните между собой звенья десятым стержнем. Затем повторите эти действия для остальных 3 колес, после чего установите их на шасси. См. Рис. 7.1.

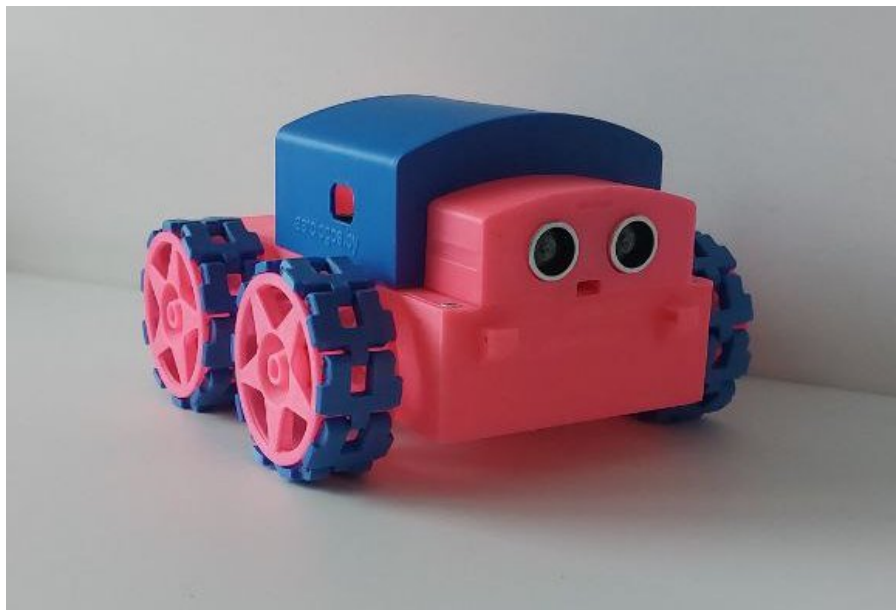


Рис. 7.1: Робот на колесной базе

7.4 Робот на гусеничной базе

Разделите звенья и пальцы на 2 части - по 20 звеньев и 20 стержней в каждой. Соедините каждую группу стержнями таким образом, чтобы получилось 2 замкнутые гусеницы.

Аккуратно установите колеса на задние валы. Возьмите колесо для переднего вала, наложите на него гусеницу, затем наложите противоположную сторону гусеницы на заднее колесо и только после этого надевайте переднее колесо. Таким образом вы легко можете снимать и надевать гусеницы, избегая риска сломать задние валы.

Повторите эти же действия для второй группы колес и гусениц. См. Рис. 7.2.

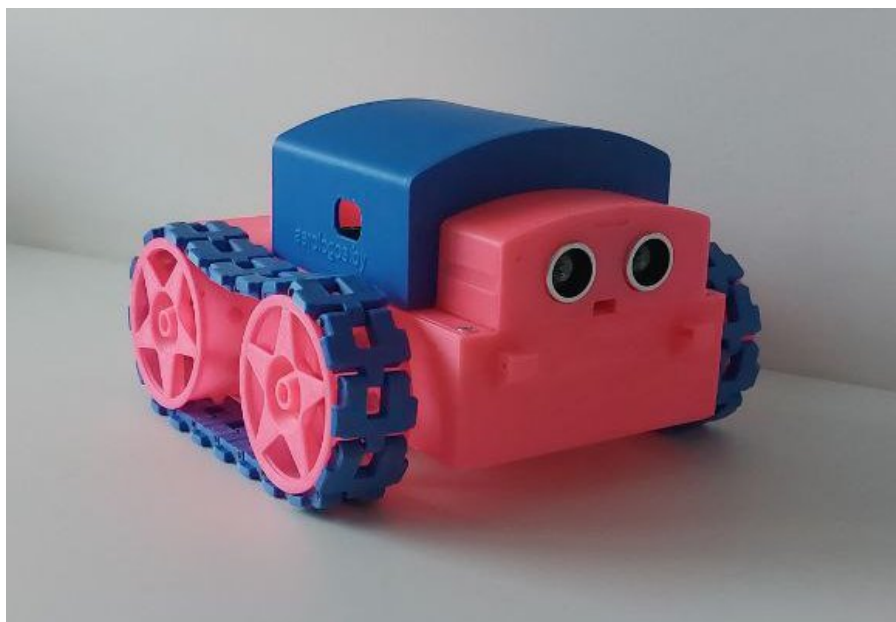


Рис. 7.2: Робот на гусеничной базе

7.5 Установка аккумулятора

Подсоедините аккумулятор, уложите его внутрь отсека и закройте крышкой.

Теперь Ваш робот полностью собран и готов к эксплуатации.

Глава 8

Продвинутые алгоритмы движения

8.1 Движение с поворотами

Проверим, умеет ли робот поворачивать.

Откройте снова файл *ardu_bot.ino* в Arduino IDE, закомментируйте строки с подключением библиотеки и создания объекта *movement_algorithm_straight*.

Раскомментируйте группу строк для работы с алгоритмом *movement_algorithm_turner*.

Убедитесь, что файл компилируется, нажав на соответствующую кнопку в Arduino IDE.

Подключитесь к роботу с компьютера через синий провод и нажмите на кнопку загрузки прошивки.

Дождитесь подтверждения и отсоедините провод.

Поставьте робот на пол и включите - он должен начать двигаться по квадратной траектории, проезжая некоторое расстояние и делая поворот. И так до тех пор пока не будет выключен.

Согласен, не очень интересно, зато мы убедились, что он

умеет поворачивать.

8.2 Движение с поиском пути

Попробуем что-то более приближенное к реальности.

Закомментируйте строки с подключением библиотеки и создания объекта *movement_algorithm_turner*.

Раскомментируйте группу строк для работы с алгоритмом *movement_algorithm_finder*.

Убедитесь, что файл компилируется, нажав на соответствующую кнопку в Arduino IDE.

Подключитесь к роботу с компьютера через синий провод и нажмите на кнопку загрузки прошивки.

Дождитесь подтверждения и отсоедините провод.

Поставьте робот на пол и включите - он должен начать двигаться, как живое существо. Встречая препятствия, он будет объезжать их. Застрыв - делать ход назад и менять направление.

Исполнение одного и того же алгоритма может отличаться для колесной и гусеничной платформ. Например, движение по скользкому ламинату тяжело дается роботу на колесах, так как он постоянно пробуксовывает. А вот в гусеничном - таких проблем нет. По хорошо скошенной траве гусеницы тоже выигрывают, так как лучше цепляются за поверхность.

Но и в колесной базе есть свои преимущества. Например, двигаясь по бетонной плитке и асфальту робот на колесах будет ездить также уверенно, как и на гусеницах, однако проезжает он дольше, так как будет потреблять меньше энергии. К тому же в образе робота на колесах присутствует свой уникальный шарм и кому-то он больше по вкусу.

В общем советуем вам попробовать оба варианта и выбрать тот, который понравится больше или менять их периодически.

Глава 9

Погружение в детали

После большой работы над сборкой и прошивкой робота, Вам вероятно захочется придумать свой собственный алгоритм движения. Для этого вам потребуется углубиться в детали уже существующих алгоритмов, а затем изменить какой-то из них под себя или же создать новый.

Рекомендуем пересмотреть листинги уже озвученных ранее алгоритмов в той же последовательности.

```

README.md  ardu_botino  movement_algorithm.h  movement_algorithm_straight.h  movement_algorithm_turner.h  movement_algorithm_finder

4  #include "movement_algorithm.h"
5  #include "distance_meter_sonar.h"
6  #include "chasis_two_front_wheels.h"
7
8  class movement_algorithm_straight : public movement_algorithm {
9  public:
10     void init() {
11         _distance_meter.init();
12         _chasis.init();
13
14         Max_distance = _distance_meter.get_max_distance();
15         _chasis.stand_still();//CRITICAL! used to stop last chasis move command when board is reloaded by itself.
16     }
17
18     void move() {
19         //delay(500); //simplifies debugging
20         _chasis.stand_still();//CRITICAL! used to stop last chasis move command.
21
22         int front_distance = _distance_meter.get_front();
23         _profiler.print_status("front_distance", front_distance);
24         _profiler.print_status("Prev_front_distance", Prev_front_distance);
25
26         int dist_to_move = front_distance * 0.5; // 50% of measured distance to prevent collisions
27         if (dist_to_move > 10) {
28             _profiler.print_status("go_forward", dist_to_move);
29             _chasis.go_forward(dist_to_move);
30         }
31         else {
32             _profiler.print_status("go_backward", Backward_distance);
33             _chasis.go_backward(dist_to_move);
34         }
35     }
36 }

```

Рис. 9.1: Листинг алгоритма для движения прямо и назад.

Обратите внимание на объекты `_chasis` и `_distance_meter`. К ним применяется тот же подход с абстракциями, который мы обсуждали ранее.

9.1 Шасси

Класс `_chasis_two_front_wheels` объекта `_chasis` реализован для управления шасси с 2 жестко зафиксированными передними колесами. Эта конфигурация диктует нам, как организовать, например, повороты вправо или влево. Если бы нашим шасси была платформа на 2 или 4 ногах, то и реализация была бы продиктована такой конфигурацией, а также компонентной базой, используемой для нее.

Важно то, что вне зависимости от внутренних особенностей конкретных шасси, они все реализованы согласно общему контракту, определенному в классе *chasis*, в котором есть функции движения вперед/назад, поворотов вправо/влево, разворотов и т.д.

```
/*
 * Describes the baseline contract for controlling the chasis of the robot
 */
class chasis {
public:
    virtual void init();
    virtual void stand_still();
    virtual void go_forward(double distance);
    virtual void go_backward(double distance);
    virtual void go_left();
    virtual void go_right();
    virtual void go_forward_left();
    virtual void go_forward_right();
    virtual void turn_left();
    virtual void turn_right();
    virtual void turn_around();
};
```

Рис. 9.2: Базовый контракт для шасси

То-есть наверху, где мы пишем алгоритм движения робота, мы не думаем про то, какое шасси используется, а думаем только о логике перемещения. Это очень удобно и значительно сокращает умственную нагрузку при проектировании как функциональности шасси, так и логики движения.

9.2 Измерение расстояния

Тоже самое применено в классе *_distance_meter_sonar* объекта *_distance_meter*. Он опирается на использовании ультразвукового дальномера *HC-SR04*, реализуя контракт ба-



Рис. 9.3: Ультразвуковой дальномер HC-SR04

зового абстрактного класса *distance_meter*. Дальномер *HC-SR04* может измерять расстояние до 4 метров.

Стоит отметить, что сейчас в классе не реализованы методы *get_left* и *get_right* потому что глаза у Вашего робота зафиксированы и не могут поворачивать для определения расстояния справа или слева. Чтобы это было возможно дальномер необходимо устанавливать на сервоприводе и включать в реализацию функциональность управления сервоприводом.

Измерение расстояния можно организовать и другими средствами. Например с использованием инфракрасного датчика расстояния GP2Y0A02YK0F. Об этом стоило упомянуть, чтобы показать возможность масштабирования для измерения расстояния, но данная процедура выходит за рамки данного руководства.

В случае если Ваш интерес выйдет за границы сборки робота Бобик, вы сможете пользоваться нашими программными наработками в области управления роботом и создать свою уникальную платформу.

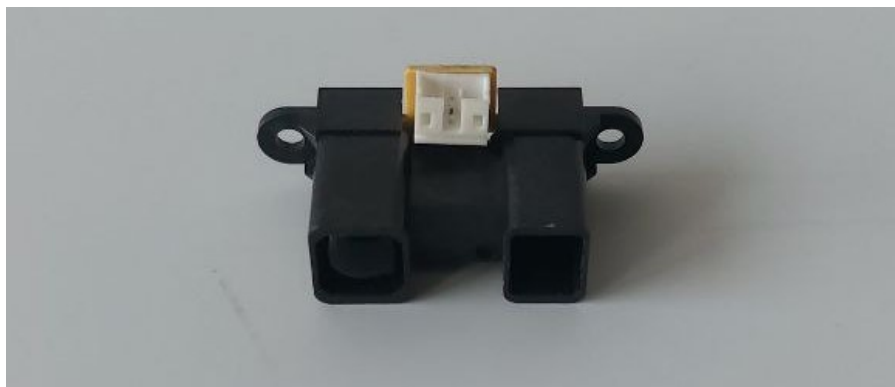


Рис. 9.4: Инфракрасный датчик расстояния GP2Y0A02YK0F

Глава 10

Отладка

Предположим Вы испытали все наши алгоритмы и Вам захотелось создать свой на базе класса *chasis*. Вам кажется, что все сделано правильно, но робот ведет себя не так, как ожидается.

Можно попробовать перечитать листинг своего алгоритма и найти ошибку, но это обычно отнимает много времени и не всегда удается сразу понять в чем дело.

Мы рекомендуем воспользоваться помощником, определенным в классе *profiler*, который реализован на базе стандартного инструмента отладки для Arduino алгоритмов *Serial Monitor*.

Перейдите в головной файл *ardu_bot.ino* и убедитесь в том, что строка с *_profiler.init()*; не закомментирована. Она отвечает за активацию *Serial Monitor*.

Затем в файле вашего алгоритма либо в любом другом файле, где вы подозреваете находится ошибка, создайте объект написав *profiler _profiler*;

После этого воспользуйтесь вызовом метода *_profiler.print _status* как показано на Рис. 10.1, где первым параметром идет строковой литерал, а вторым числовое значение.

Поставьте такие вызовы везде, где необходимо в реальном времени посмотреть на значения переменных и загрузите код

```
void move() {  
    //delay(500); //simplifies debugging  
    _chasis.stand_still();//CRITICAL! used to stop last chasis move comma  
  
    int front_distance = _distance_meter.get_front();  
    _profiler.print_status("front_distance", front_distance);  
    _profiler.print_status("Prev_front_distance", Prev_front_distance);  
}
```

Рис. 10.1: Использование профайлера

на борт робота. Затем, не вытаскивая провод, запустите робот и нажмите на кнопку включения/выключения *Serial Monitor*, чтобы внизу появилась соответствующая вкладка.

Как только Вы все это сделаете, внизу начнет появляться информация, при помощи которой вы сможете понять, все ли происходит так, как было запланировано, или есть ошибки. Если все в порядке, попробуйте настроить профилирование в других частях программы и искать проблемы там. Если проблема нашлась, устраните ее и залейте новый код.

Важно! Если у вас не получается загрузить код на борт робота, то возможно дело в окне вывода Serial Monitor, который остался активен после того, как Вы вытащили провод. Это очень частая проблема, не зная о которой можно потратить кучу времени зря. Так что всегда перед загрузкой кода убедитесь что окно вывода Serial Monitor выключено.

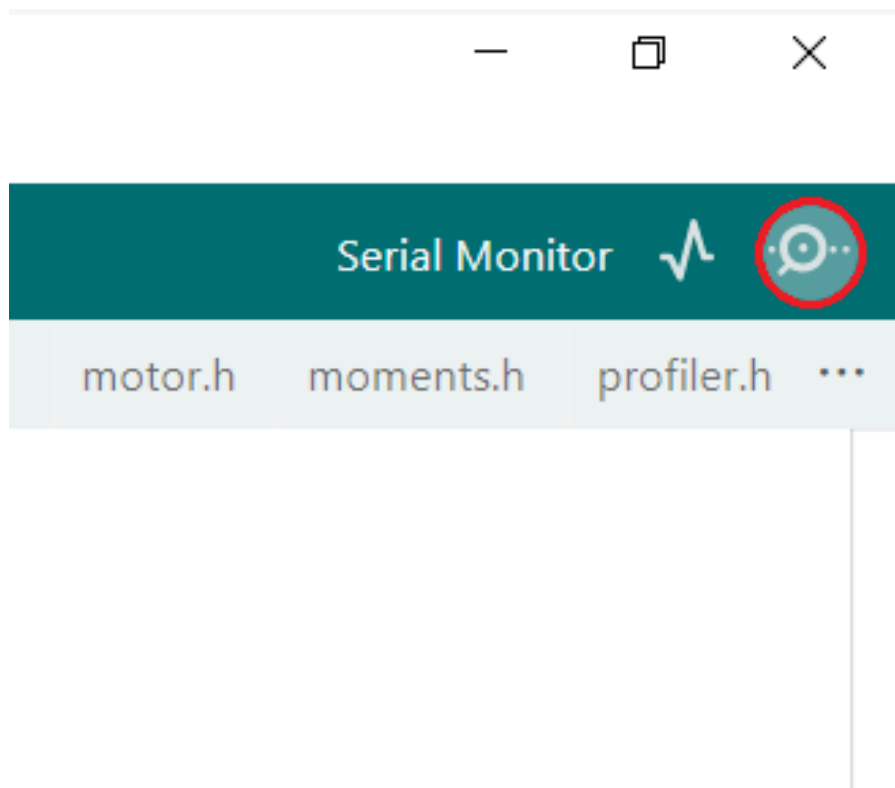


Рис. 10.2: Включение Serial Monitor

```
20
21 void loop()
22 {
23     movement_algorithm.move();
```

Output Serial Monitor X

Not connected. Select a board and a port to connect automatically.

```
_last_recorded_dist: 72
front_distance: 72
Prev_front_distance: -1
go_forward: 36
_last_recorded_dist: 61
front_distance: 61
Prev_front_distance: -1
go_forward: 30
_last_recorded_dist: 84
front_distance: 84
Prev_front_distance: -1
go_forward: 42
```

Рис. 10.3: Вывод Serial Monitor

Глава 11

Рекомендации

Если у Вас появится желание развить возможности собранного робота или создать свой, Вам потребуются более глубокие знания и понимание, как использовать и программировать периферию Arduino.

Мы рекомендуем на выбор следующие материалы для самостоятельного изучения:

- Михаил Момот - Мобильные роботы на базе Arduino
- Саймон Монк - Программируем Arduino
- Юрий Менщиков – Ардуино на пальцах.

Глава 12

Заключительное слово

Вы закончили краткий вводный курс по программированию роботов на базе контроллера Arduino UNO в теле робота Бобик. Это лишь малая часть тех знаний, которыми изобилует это направление. Нашей задачей было познакомить Вас с основами и разъяснить, как их применить для сборки, настройки и программирования робота, который Вы приобрели у нас. Надеемся у нас это получилось.

Успехов Вам в дальнейшем изучении...

Инструкция по сборке робота

Б О Б И К

АЭРОЛОГОС